ANNEXURE – V

THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY, PATIALA

Department of Electronics & Communication Engineering

PROJECT SEMESTER: 2025-26 Even Semester (from January 2026)

Internship Student MIDWAY REPORT

Date of Visit: 13th April, 2026

Roll No. & Name of Student: 102206125, Varnit Raj Singh

Name of Organization & Address: Excitel Broadband, 2nd & 3rd Floor, Plot No. 48, Okhla Industrial Estates, Phase-III, New Delhi – 110020

Phone No: 7626910250 **Email:** vsingh1_be22@thapar.edu

Name of Industry Mentor: Chirag Bhatnagar **Designation:** NOC-L1 **Phone No:** 7977921321
Email: chirag.bhatnagar@dl.excitel.in

Topic/Title of Project: Machine Learning Model Development and Deployment with Full-Stack Web Interface

Type of Project: Software Development / Machine Learning / Full-Stack Web Application

1. Overview of Work Completed

The internship began with building a solid technical foundation across multiple domains before progressing into the core ML development work. The first phase covered everything from language basics and infrastructure setup to machine learning theory and hands-on model implementation. The following sections describe each area of work completed up to this midway point.

1.1 Foundation — Python, NumPy & Pandas

Since Python was not the primary language going in, the first step was getting genuinely comfortable with it — not just syntax, but the way Python is actually used in data and ML workflows. This covered:

- Python fundamentals: syntax, loops, functions, OOP concepts, and file handling.
- NumPy: arrays, matrix operations, broadcasting, and vectorised computations that form the backbone of numerical work in Python.
- Pandas: loading datasets, cleaning messy data, handling missing values, aggregating, filtering, and transforming dataframes into a shape suitable for modelling.

Detailed notes were maintained throughout, making it easier to refer back during implementation.

1.2 Linux, Networking & Virtual Machines

Working in a real server environment requires comfort with Linux, and this was addressed early on:

- Linux (Ubuntu Server): navigated the file system, managed permissions, wrote shell scripts, and ran server operations directly from the terminal.

•

Networking Basics: went through a structured playlist covering switches, routers, IP addressing, subnets, and communication protocols — directly relevant when containers and APIs need to talk to each other.

- Virtual Machines: explored how hypervisors work, spun up a VM environment, and used it as a sandbox for testing without affecting the host system.

1.3 Version Control, SSH & Automation

A professional development environment was set up from day one to ensure organised and traceable work:

- GitHub Repository: connected a remote repo and practised everyday version control — committing changes, pushing code, and managing branches.
- SSH Key Setup: generated an SSH key pair and linked it to GitHub and the remote server for secure, password-less authentication.
- Cron Jobs: learned task scheduling in Linux. A practical task was set up — a cron job running every 3 minutes that writes its output to an output.log file, demonstrating real-world automation.

1.4 Docker & Containerisation

Docker was covered in depth as it will be central to the final deployment phase:

- Understood what containers are and how they differ from virtual machines.
- Built Docker images, wrote Dockerfiles, and managed container lifecycles.
- Explored docker-compose for managing multi-service setups where the ML backend, API, and frontend run as coordinated containers.
- Thought through how a trained ML model gets packaged and deployed inside a container, making it portable and production-ready.

1.5 Machine Learning — Theory and Implementation

Before writing any model code, time was invested in understanding the theory behind what the models are actually doing. The study covered regression (predicting continuous values), binary and multi-class classification (decision boundaries, loss functions), and a brief introduction to image problems.

Following the theoretical grounding, all models were implemented entirely from scratch in Python — no scikit-learn shortcuts for the core model logic. The goal was to genuinely understand every step under the hood:

a) Models Implemented:

- Regression: Linear Regression + MLP with manual backpropagation
- Binary Classification: Logistic Regression + MLP • Multi-class Classification: Softmax Regression + MLP

b) Implementation Details:

- Dataset selected, described, and justified with a primary evaluation metric.

•

EDA performed — distributions explored, correlations identified, outliers flagged, and data cleaned.

- Data split (80-20), missing values handled, categorical columns encoded, and features scaled.
- Baseline model: weights initialised, forward pass implemented, loss computed, gradients calculated, weights updated via $w = w - lr \times grad$.
- MLP: multi-layer network with ReLU activations, full forward and backward propagation coded manually following the chain rule.

c) Evaluation Metrics Used:

- Regression: MSE, RMSE, MAE, R^2
- Classification: Accuracy, Precision, Recall, F1-Score, Confusion Matrix

d) Visualisations Generated:

- Training loss curves comparing Baseline vs. MLP model convergence.
- Performance comparison bar charts across all models.
- Domain-specific exploratory plots.

1.6 RAG Pipeline — LangChain & PDF Parser

A Proof of Concept (PoC) for a Retrieval-Augmented Generation (RAG) pipeline was built using Python and the LangChain framework. The objective was to design a system that could extract, enrich, and structure information from PDF documents for efficient semantic retrieval.

The PDF parsing pipeline works as follows:

1. The user uploads a PDF document into the system.
2. Text is extracted from the PDF and stored in a variable called full_text_content.
3. Images are extracted from the PDF. A Python OCR tool extracts text from each image and appends it to full_text_content labelled as "OCR Text". The same images are then passed to an LLM, which generates concise descriptions — also appended as "Image Description".
4. A unified, enriched representation of the document is returned — combining original text, OCR output, and LLM-generated image descriptions.

After the pipeline produces the full text, a recursive text chunker splits it into semantically meaningful chunks. Each chunk is converted into vector embeddings and stored in a vector store, enabling similarity-based retrieval for the RAG pipeline.

2. Results and Outcomes Achieved

The following concrete outcomes have been achieved by the midway point:

- Proficiency established in Python, NumPy, and Pandas — sufficient for data-heavy ML work.
- Functional Linux + Ubuntu server environment configured and in active use.
- GitHub repository connected with SSH-based authentication; version control integrated into daily workflow.
- Automated cron job running and logging output every 3 minutes — confirmed working.
- Docker environment set up; multi-container workflows understood and tested.

- All six ML models (Linear Regression, Logistic Regression, Softmax Regression, and three MLPs) implemented from scratch with working forward pass, backpropagation, and gradient descent.
- EDA, data preprocessing, training, evaluation, and visualisation pipeline completed end-to-end.
- Functional RAG pipeline PoC built — PDF parsing, OCR integration, LLM image descriptions, chunking, embedding, and vector store indexing all operational.

3. Major Challenges Faced

- Python as a New Language: Since Python was not used prior to this internship, getting up to speed quickly while simultaneously applying it to data and ML tasks required extra effort. The approach taken was to learn by doing — writing actual code rather than just reading documentation.
- Manual Backpropagation: Implementing backpropagation through the chain rule entirely from scratch — without relying on autograd frameworks — was the most mathematically demanding part. Getting the gradient calculations right for multi-layer networks took multiple iterations of debugging.
- PDF Image Processing: Extracting images from PDFs and applying OCR reliably across varied PDF formats was more complex than anticipated. Some PDFs embed images in formats that standard libraries struggle with, requiring fallback handling.
- LLM Integration for Image Descriptions: Getting an LLM to generate consistent, concise image descriptions (rather than verbose or hallucinated ones) required careful prompt engineering and testing across different types of document images.
- Docker Networking: Configuring containers to communicate with each other (especially when the API and the ML model are in separate containers) involved working through network bridge configurations and port mapping in docker-compose.

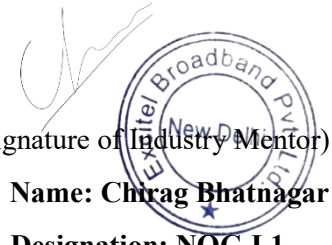
4. Innovations and Key Learnings

- The PDF pipeline combines three distinct types of information extraction (raw text, OCR from images, and LLM-generated descriptions) into a single enriched representation — this multimodal approach significantly improves downstream retrieval quality compared to text-only parsers.
- All ML models were written with modularity in mind — the weight initialisation, forward pass, loss computation, and update step are cleanly separated, making it easy to swap in different architectures without rewriting the training loop.
- Notes maintained throughout the learning phase were structured not just as summaries but as personal reference guides — capturing the "why" behind each concept, not just the "what".

5. Remaining Work to Be Completed

The following tasks remain to be completed before the end of the project semester:

- Backend API Development: Build a REST API using Django or Flask to serve the trained models — accepting input data and returning predictions in real time.
- Frontend Interface: Develop a React.js web interface where users can enter values, submit them, and receive model predictions through a clean, functional UI.
- Full Deployment: Containerise the complete application (backend + frontend + models) using Docker and deploy it as a working end-to-end system.
- Image-Based ML Problem (Time Permitting): Explore a basic computer vision problem to extend the ML work into the image domain — a brief introduction was covered in theory; practical implementation is pending.
- RAG Integration with Frontend: Connect the RAG pipeline to a query interface so users can ask questions over uploaded documents and receive context-grounded answers.
- Final Testing and Documentation: End-to-end testing of all components, fixing edge cases, and producing clean documentation for the full system.



(Signature of Faculty Mentor)

Name: Dr. Rishikesh Pandey

Designation: Assistant Professor

(Signature of Industry Mentor)

Name: Chirag Bhatnagar

Designation: NOC-L1